Python Activity-Based Learning
Case Study

49 Tuple-Based Product Catalogue Audit

Selected case study | Prepared using the topics listed in the supplied assignment.

1. Problem Statement

Maintain a product catalogue using tuples containing product ID, price, and stock quantity. Identify invalid records, calculate inventory value, and list products that need replenishment.

2. Objectives

• Store fixed product records using tuples.

• Unpack tuple values into variables.

• Calculate stock value.

• Use conditions to identify low stock.

3. Assumptions

• Each product is represented as (product_id, price, quantity).

• Price must be greater than zero and quantity cannot be negative.

• A product needs replenishment when stock is below 5 units.

• The catalogue is entered directly in the program.

4. Algorithm

1. Create a list of product tuples.

2. Initialize total inventory value.

3. For each tuple, unpack ID, price, and quantity.

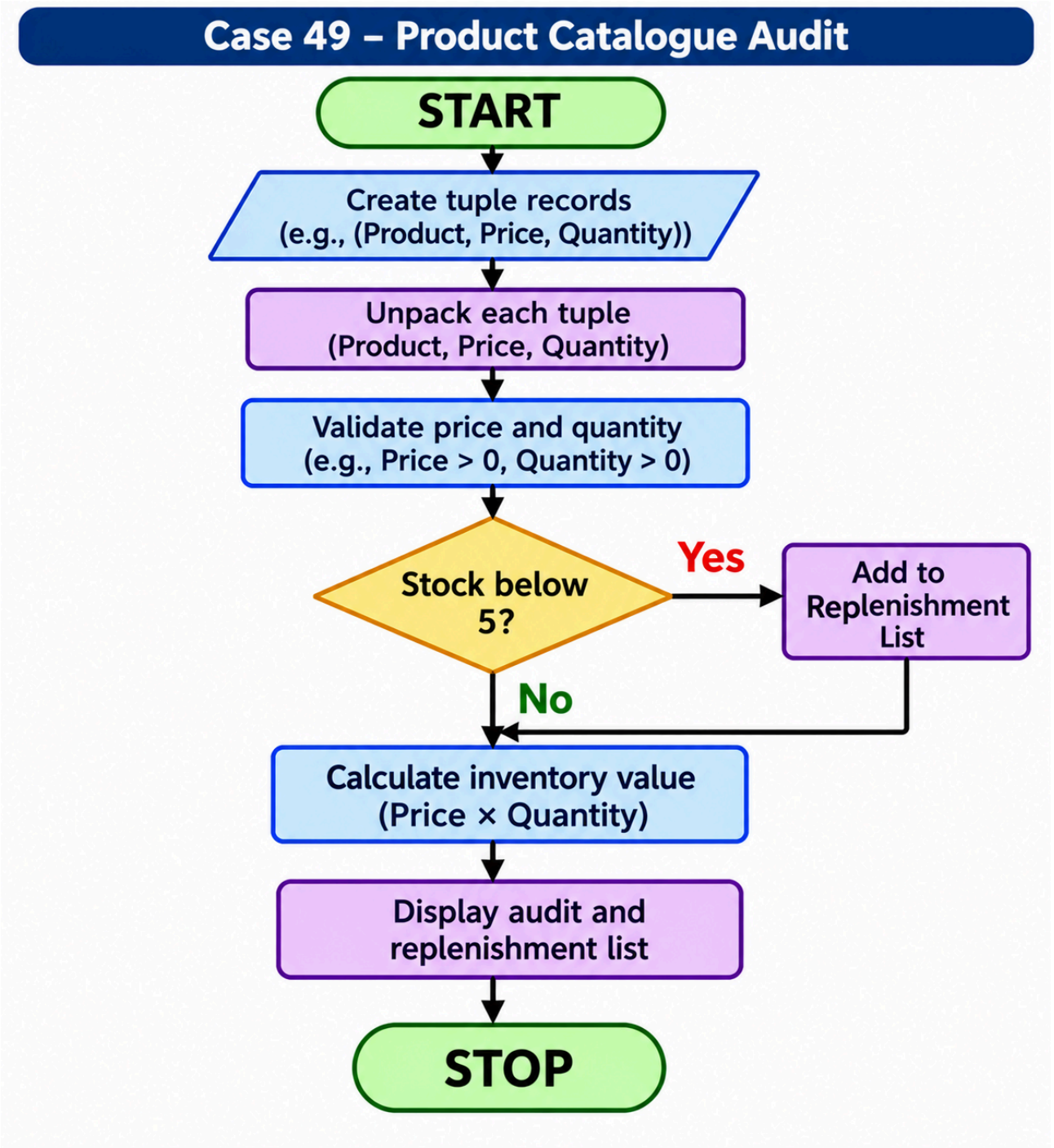
4. Check record validity.

5. For valid records, add price multiplied by quantity to inventory value.

6. If quantity is below 5, add the product to the replenishment list.

7. Display catalogue audit results.

5. Flowchart



6. Pseudocode

```
BEGIN
 catalogue ← list of product tuples
 inventory_value ← 0
 FOR each product tuple
 UNPACK id, price, quantity
 IF price > 0 AND quantity >= 0 THEN
 inventory_value += price × quantity
 IF quantity < 5 THEN add id to reorder list
 ELSE report invalid record
 END IF
 END FOR
```

```
 PRINT inventory_value and reorder list
END
```

Case Study 49 — Tuple-Based Product Catalogue Audit

7. Python Source Code

```
catalogue = [
 ("P101", 50, 10),
 ("P102", 120, 3),
 ("P103", 75, 0),
 ("P104", 200, 8)
]

inventory_value = 0
reorder_list = []

print("--- PRODUCT CATALOGUE ---")
for product in catalogue:
 product_id, price, quantity = product

 if price <= 0 or quantity < 0:
 print(product_id, "Invalid record")
 else:
 value = price * quantity
 inventory_value += value
 print(product_id, "Price:", price,
 "Quantity:", quantity, "Stock value:", value)

 if quantity < 5:
 reorder_list.append(product_id)

print("\nTotal inventory value:", inventory_value)
print("Products requiring replenishment:", reorder_list)
```

8. Test Cases

Input / Scenario	Expected result
P101 (50,10); P102 (120,3)	P102 needs replenishment; their combined stock value is ■860
Quantity 0	Valid record; needs replenishment
Price -10	Invalid record

9. Conclusion

The program uses tuples, tuple unpacking, lists, loops, and conditions to audit a simple product catalogue.

Note: Run the program in Python 3. The sample test cases are illustrative; enter the values when

prompted.